Real-Time Localization Framework for Autonomous Basketball Robots

Naren Medarametla
School of Computer Science Engineering
Vellore Institute of Technology
Chennai, India
naren.medarametla2023@vitstudent.ac.in

Sreejon Mondal
School of Electrical and Electronics Engineering
Vellore Institute of Technology
Chennai, India
sreejon.mondal2023@vitstudent.ac.in

***Abstract*— Localization is a fundamental capability for autonomous robots, enabling them to operate effectively in dynamic environments. In Robocon 2025, accurate and reliable localization is crucial for improving shooting precision, avoiding collisions with other robots, and navigating the competition field efficiently. In this paper, we propose a hybrid localization algorithm that integrates classical techniques with learning-based methods, relying solely on visual data from the court's floor to achieve self-localization on the basketball field.**

***Keywords*—Robot Localization, Autonomous Navigation, Neural Networks, Robocon**

I. Introduction

The DD-Robocon is the Indian national entry-level competition to Asia-Pacific Robot Contest (ABU Robocon). The objective of the competition is for college teams to design, build, and operate robots to complete certain tasks. The tasks vary each year and are usually based on cultural, historical, or technological themes related to the host country of the international contest. For Robocon 2025, the theme was Robot Basketball, where two robots had to play the game of basketball following specific rules. The arena consisted of a court of dimensions 15m × 8m and baskets at a height of 2.4m. Due to the nature of the game, the robots had to move quickly and unpredictably, and take shots from various positions on the court. The robots had to have robust and real-time self-localization determine it's distance from the basket. In this paper we propose a vision-based localization system based on the white field lines of the basketball court.

Section II reviews existing methods and prior research related to this work. **Section III** provides a detailed description of our proposed algorithm, approach, and model architecture. **Section IV** provides the results obtained from our experiments. **Section V** discusses previous approaches which failed. **Section VI** evaluates the accuracy of our approach and discusses potential directions for future work.

II. Related Work

In vision-based self-localization for dynamic field sports, researchers have explored complementary lines of work that balance geometric modeling, probabilistic estimation, and learned perception. Liu et al. [4] introduced a pose–point matching method for omnidirectional cameras in which white field lines are detected from brightness changes, corrected for lens effects, and matched against a field model to recover position and heading in real time with errors reported on the order of tens of centimeters. Ribeiro et al. [9] accelerated global relocalization by precomputing a distance map of the pitch and then searching the error surface between observed line points and this map under a known heading, which makes frequent global updates practical when calibration and line extraction are reliable. Watanabe et al. [11] framed model matching as a search problem and optimized the robot pose with a genetic algorithm driven by omnidirectional white–line features, which helps in scenes with sparse or ambiguous cues while requiring careful parameterization to remain real time.

Lu et al. [5] reported a robust real–time pipeline based on omnidirectional vision that emphasizes efficient feature extraction and stable operation during matches, though performance naturally depends on lighting and occlusions typical of indoor arenas. On humanoid platforms, Tian et al. [10] adapted Monte Carlo Localization to fisheye optics and bipedal gait by designing a motion model for oscillatory head movement and a vision model for field landmarks, together with a resampling strategy that stabilized estimates on limited hardware. Fusion-based designs combine complementary sensing to limit drift and maintain responsiveness, Ismail et al. [2] fused encoder and gyro odometry with omnivision in a particle filter so that fast motion updates are anchored by drift-free visual corrections even on symmetric fields.

More recent work uses inertial constraints to stabilize vision before geometric reasoning, Nadiri et al. [8] filtered IMU orientation and applied inverse perspective mapping to obtain a bird's-eye view from which field markers provide consistent distances on approximately planar surfaces. Learning-based perception is widely used to localize robots relative to other agents rather than only to the field, Luo et al. [6] detected

robots with a convolutional detector and estimated 3D positions through RGB–D registration on embedded hardware, improving robustness to appearance variation at the cost of additional calibration and power.

Table I: Comparison of Related Works

Author & Year	Camera Type	Methodology	Performance	Limitations
Liu et al, 2009	Omnidirectional	Extracts white-line feature points from brightness changes and matches them to model "pose points" for position & heading estimation.	Achieves approx. 20 fps with approx. 20 cm mean error, robust to changes in goal color.	Relies on good visibility of white lines, requires careful distortion correction.
Ribeiro et al, 2016	Omnidirectional	Uses precomputed distance maps of field lines, matches observed line points to the least-error grid cell given a known heading.	Allows for very fast global localization and frequent updates.	Needs an accurate heading and precise field calibration, performance degrades with poor line detection.
Watanabe et al, 2020	Omnidirectional	Combines white-line detection with a Genetic Algorithm (GA) to optimize the pose model match.	Provides robust performance even with sparse line cues and enables an effective global search.	Requires tuning the GA for real-time performance, has a higher computational cost than analytic matchers.
Luo et al, 2019/2020	RGB + Kinect v2 Depth	A two-stage process where YOLOv2 detects robots in RGB, and then Kinect depth data is used to recover their 3D positions.	High Mean Average Precision (mAP), robust against changes in jersey color.	Requires an active depth sensor (limiting it to indoor use), cross-sensor calibration, and has extra power demands.
Lu et al, 2013	Omnidirectional	Employs a robust omnidirectional vision pipeline for feature extraction and localization.	Has demonstrated real-time robustness in practical applications.	Sensitive to lighting conditions and occlusion.
Tian et al, 2010	Fisheye Lens	Implements a particle filter (Monte Carlo Localization) with vision and motion models, handling distortion and motion oscillation.	Manages non-Gaussian noise and multiple hypotheses, has been proven on hardware.	Incurs a higher computational load, can suffer from pose aliasing in symmetric fields.
Ismail et al, 2012	Omnidirectional + Odometry	Fuses data from odometry (which is fast but drifts) and omnidirectional vision with MCL (which is drift-free but slower).	Delivers accurate and responsive localization with low latency (approx. 1.6 ms).	Requires careful time synchronization, MCL remains compute-heavy with sparse data.
Nadiri et al, 2025	Omnidirectional + IMU	Uses an IMU-stabilized camera with a Bird's-Eye View (BEV) transformation for marker detection.	Reduces mean error by approximately 9 cm, the BEV simplifies distance measurement.	Demands reliable IMU calibration, sensitive to errors in camera setup.

Unlike most of the reviewed approaches, which rely heavily on omnidirectional or fisheye optics to maximize field-of-view for white-line detection, our method uses a regular monocular camera while still achieving reliable self-localization. This choice reduces the need for specialized distortion correction pipelines, and makes the approach more adaptable to standard camera modules already used in many robotics platforms.

III. Methodology

Our approach is a two-step process that begins with **Preprocessing** the image, followed by passing it to the model for **Inference**.

A. Preprocessing

The input image having dimensions $(640 \times 480 \times 3)$ is converted from the RGB color space to the HSV color space, then the white regions are masked out using two predefined HSV ranges. The image is downsampled through a radial scan, flattened, and finally passed through the neural network. Figure I shows the preprocessing pipline and Algorithm 1 gives the implementation of the downsampling algorithm.

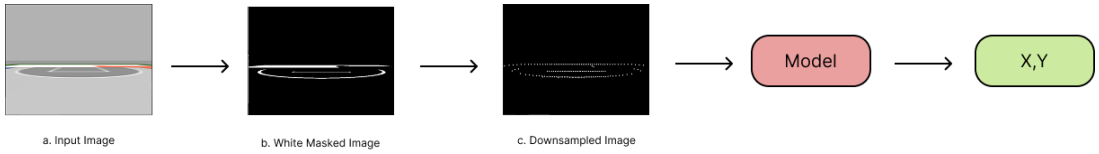
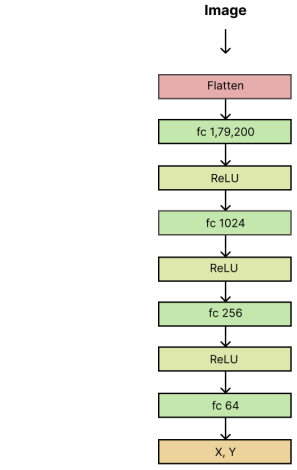


FIGURE I: PREPROCESSING PIPELINE

Algorithm 1 Downsampling

Input: Image
$H \leftarrow$ Image.height
$W \leftarrow$ Image.width
$R \leftarrow$ Black Image
for angle $\leftarrow$ 0 to 180 step 2 **do**
  $lastPixel \leftarrow 0$
  $cx \leftarrow$ W / 2
  $cy \leftarrow$ H
  for d $\leftarrow$ 0 to max(H, W) **do**
    $x \leftarrow cx + d \times \cos(angle)$
    $y \leftarrow cy - d \times \sin(angle)$
    if $0 \leq x <$ W **and** $0 \leq y <$ H **then**
      $pixel \leftarrow Image[y][x]$
      if lastPixel = 255 and pixel $\neq$ 255 **then**
        $R[y][x] \leftarrow 255$
      end if
      $lastPixel \leftarrow pixel$
    end if
  end for
end for
Return R



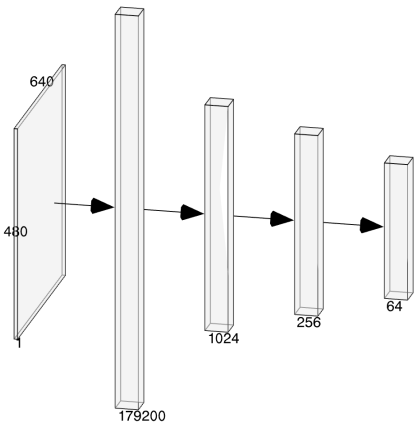


FIGURE III: ARCHITECTURE DIAGRAM

B. Model Architecture

The proposed model is a feedforward neural network consisting of a flattening layer followed by four fully connected layers. The first 200 pixels from the top of the image are removed to reduce the input size, as they do not carry useful information and the pixel values are scaled down to the range [0, 1].

The resulting image with dimensions (640 × 280 × 1) is flattened into a vector of size 1,79,200 and passed through the first linear layer, followed by a ReLU activation function. The subsequent layers have sizes 1024, 256, and 64, each followed by a ReLU activation. The final layer outputs a 2-dimensional vector representing the predicted x and y positions of the robot.

The rationale for using this relatively simple model lies in the simplicity of the input images and to reducing inference time.

C. Dataset

A digital twin of the robot was created and simulated in a replica of the Robocon 2025 arena using the Gazebo [1] simulator with the help of ROS2 [7].

The robot was driven through the arena capturing images and corresponding x, y coordinates. A TimeSynchronizer was used to synchronize the frame header and the position header. A total of 6283 images were captured and split into training and test datasets with a 0.9 to 0.1 ratio. Care was taken to include every part of the arena for an unbiased dataset.

D. Simulation

The simulation environment was designed to closely replicate the physical conditions of the target field. A detailed

world map of the area was created to serve as the operating environment, and a complete model of the robot was developed using the Unified Robot Description Format (URDF) [13].

The simulation was implemented in Gazebo, where the robot's onboard camera was modeled as a virtual sensor using Gazebo camera plugins to replicate image capture. The robot was teleoperated within the environment using a joystick, leveraging the `joy` package available in ROS 2 Humble, which enabled real-time manual control and data collection across different positions and orientations in the simulated space. The images captured in Gazebo were sent back to ROS2 where the model would process them and predict the coordinates. Figure IV illustrates the block diagram

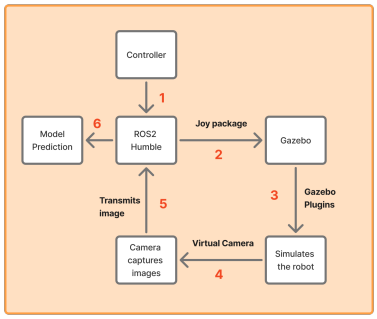


Figure IV: Blcok Diagram

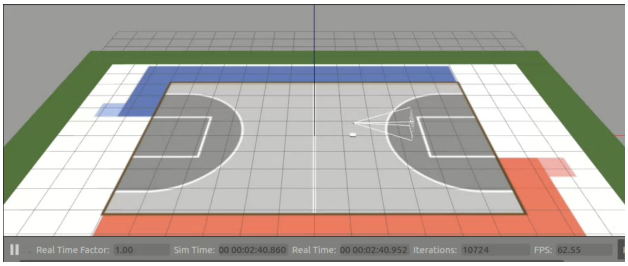


Figure V: Gazebo simulation

E. *Training*

The model was trained for 15 epochs using an Adam optimizer [3] with an initial learning rate of 10^{-4} and a Mean Squared Error (MSE) cost function in batches of size 8.

IV. Results

Figure VIII depicts the loss at each epoch throughout the training process. 8 independent images at different points of the court were again captured from the simulation and fed into the model. Figure X shows the plot between the ground truth and the prediction made by the model for these 8 images.

Table II illustrates the x and y losses for a corresponding number of test iterations. Figure VI and Figure VII show the graphical representation of this loss.

A Pearson correlation test was performed to determine if the prediction loss was dependent on the number of test iterations. For the x loss, the test yielded a p-value of 0.40. Since this value is well above the significance level of 0.05, we conclude that there is no statistically significant evidence of a relationship, indicating that the x loss is independent of the number of iterations. In contrast, the test for the y loss yielded a p-value of 0.01. This value is below 0.05, indicating a statistically significant negative correlation where the loss tends to decrease as iterations increase. A detailed breakdown of these calculations is provided below.

The code for the paper can be found in the following repository: https://github.com/NarenTheNumpkin/Basketball-robot-localization

Table II: Prediction Loss

S.No	Iterations	x loss (m)	y loss (m)
1	15	0.1375	0.1950
2	25	0.1639	0.1861
3	35	0.1498	0.1717
4	100	0.1611	0.1691
5	200	0.1464	0.1519
6	300	0.1412	0.1579
7	400	0.1394	0.1605
8	500	0.1376	0.1588
9	600	0.1401	0.1628
		0.1463	**0.1506**

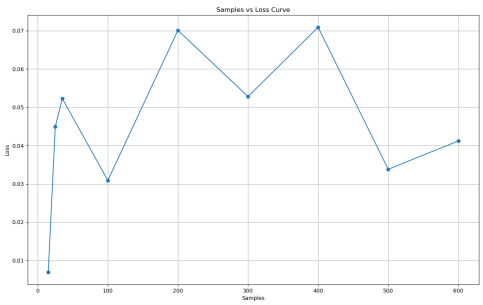


Figure VI: x loss vs iterations

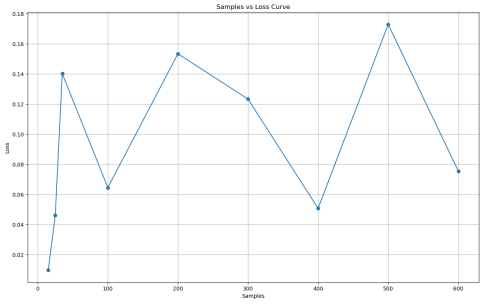


Figure VII: y loss vs iterations

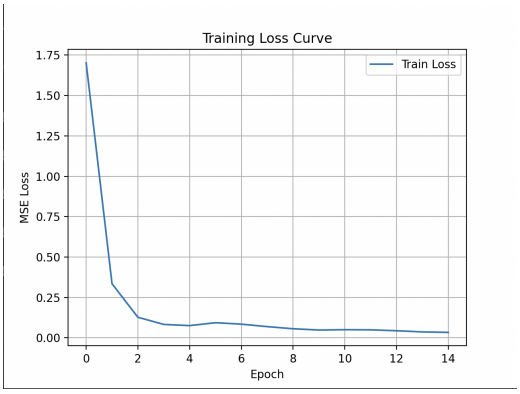


FIGURE VIII: LOSS CURVE

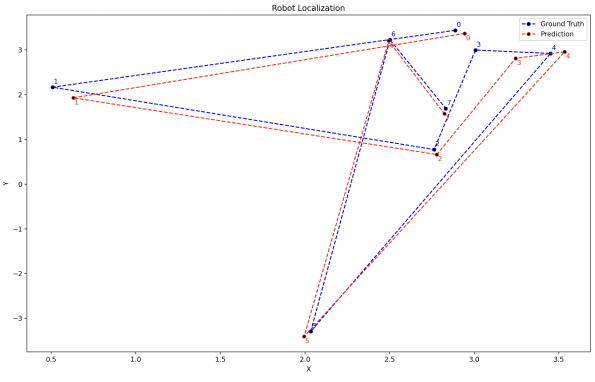


FIGURE IX: GROUND TRUTH VS PREDICTION

A. *t-test for Correlation Significance*

A t-test was used to determine the statistical significance of the Pearson correlation coefficient (r). The test evaluates the null hypothesis (H_0) that there is no linear relationship between the variables. The t-statistic is calculated using the formula:

$$t = r\frac{\sqrt{n-2}}{\sqrt{1-r^2}}$$

where n is the number of data pairs and $n-2$ gives the degrees of freedom (df). The calculated t-statistic is then compared to a critical t-value from the t-distribution for a given significance level ($\alpha = 0.05$) and df. If the calculated t-statistic is more extreme than the critical value, H_0 is rejected.

1. *Iterations vs. x loss*

For this test, we used 9 data pairs, obtained a Pearson correlation of $r = -0.32$, and computed the degrees of freedom as $df = 7$ $(n-2)$. The t-statistic is calculated as:

$$t = -0.32\frac{\sqrt{9-2}}{\sqrt{1-(-0.32)^2}} = \frac{-0.8467}{0.9474} \approx -0.89$$

The two tailed critical t-value for $\alpha = 0.05$ and df=7 is ± 2.365. Since our calculated t-statistic of -0.89 is greater than -2.365, we fail to reject the null hypothesis. The corresponding p-value is 0.40. This confirms there is no statistically significant relationship.

At $\alpha = 0.05$ there is insufficient evidence of a linear association between iterations and x loss. The observed correlation (r = -0.32) is small and likely due to sampling variability rather than a true effect.

B. *Hardware Performance*

TABLE III: MODEL PERFORMANCE ON TARGET HARDWARE

Metric	Raspberry Pi 4	Jetson Nano	GeForce RTX 4060
Inference Time (ms)			93 ms
Frames Per Second (FPS)			11
CPU Usage (%)			12% (CPU)
Memory Usage (MB)			2.31 GB
Power Consumption (W)*			˜85W

C. *Explainable AI*

To better understand the decision-making process of our model, we employed the Integrated Gradients method [12] to attribute the model's predictions to specific pixels in the input image. Integrated Gradients is a gradient-based attribution technique that satisfies desirable axioms such as sensitivity and implementation invariance. It computes feature attributions by integrating the gradients of the model's output with respect to the inputs, along a straight-line path from a baseline (typically a black image) to the actual input. This allows us to highlight the regions of the input that most influenced the model's prediction, thereby providing interpretability for both correct and incorrect decisions.

Mathematically, the attribution for the i-th input feature is computed as:

$$\text{IntegratedGradients}_i(x) = (x_i - x_i') \times \frac{1}{m}\sum_{k=1}^{m}\frac{\partial F\left(x' + \frac{k}{m}(x-x')\right)}{\partial x_i} \quad (1)$$

where x is the input, x' is the baseline input, and F is the model's output function.

This technique helped us identify which parts of the scene were most important for the model's localization decisions. In our experiments, the attributions often concentrated on regions containing field lines and other spatial cues, suggesting that the model relies heavily on geometric features.

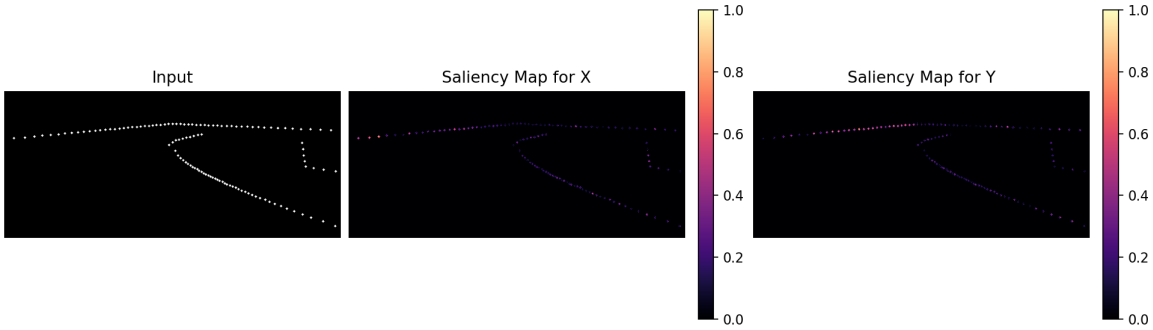


Figure X: Saliency Maps

V. Failed Approaches

In our earlier methodology, we employed an odometry-based approach for the self-localization of the robots. This system integrated wheel encoders and an Inertial Measurement Unit (IMU), complemented by a TF-Luna LiDAR sensor to estimate the depth between the robot and the basket. However, the TF-Luna sensor demonstrated a limited effective range of approximately 3 to 4 meters and exhibited suboptimal accuracy within this range. Additionally, a significant challenge arose due to the absence of a clearly distinguishable object or surface on the basket, which hindered reliable depth estimation.

In the subsequent approach, we employed an object detection model based on YOLOv8n [14], trained on a custom dataset to detect the basket. This model was utilized not only for basket detection but also for correcting the robot's yaw orientation to ensure it consistently faced the target. To estimate the horizontal distance from the basket, we defined discrete zones, with each zone representing a specific distance range. The corresponding zone, selected manually by the pilot based on visual assessment, was then used in conjunction with the detected orientation to determine the appropriate RPM for the shooting mechanism. While this approach facilitated a simplified method for distance estimation, it was inherently dependent on manual selection and coarse approximations, which could lead to reduced accuracy and consistency.

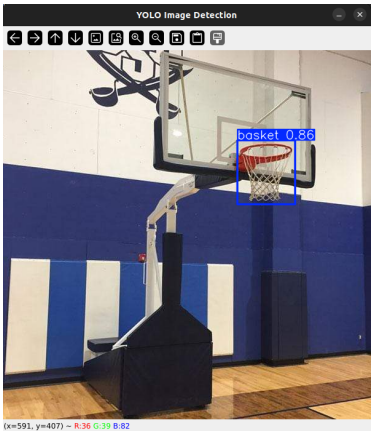


Figure XI: basket detection

We improved upon the object detection approach by using the MiDaS [15] (Multiple Depth Estimation Accuracy with Single Network) deep learning model. MiDaS is a designed for monocular depth estimation, i.e., predicting depth from a single image. It uses a ResNet-based encoder-decoder architecture where the encoder extracts features from the input image and the decoder upsamples these features to produce a depth map. The approach combines the YOLOv8 model with the MiDaS depth estimation framework for relative depth prediction. The pipeline begins by loading an input image, followed by inference using the YOLO model to localize the basket and identify its center. Simultaneously, MiDaS processes the same image to generate a dense relative depth map. Once the basket is detected, the relative depth at the center of the bounding box is extracted from the depth map. An interactive calibration step is used to relate MiDaS's relative depth to real-world distance. The user is prompted to enter known ground-truth distances for three distinct positions. These pairs of relative depth and real distance are then used to fit a linear regression model, enabling real-time conversion of relative depth to metric distance. During tracking mode, the system periodically predicts the basket's distance using the learned regression model and overlays the result on the original image alongside the visualized depth map. The output is displayed in a side-by-side format showing both RGB and color-mapped depth views for easier interpretation.

Figure XII: basket depth estimation

A key limitation of this approach was its reliance on prior calibration, which required manually measuring the distance between the robot and the basket at least three

times before accurate predictions could be made. In response to these constraints, we developed the current method presented in this paper, building upon insights gained from the previous techniques.

VI. Conclusion

In this work, we presented a hybrid localization framework for autonomous basketball robots competing in the Robocon 2025 arena. Our approach relies solely on floor images processed through a lightweight feedforward neural network. By preprocessing the images to highlight salient floor features and using a simple architecture, we achieved an average prediction error of approximately 0.06 meters, demonstrating the potential of learning-based localization methods. This result shows that even with minimal sensor inputs and modest model complexity, it is possible to achieve sufficiently accurate localization. Future improvements could focus on integrating other sensors such as IMU, wheel odometry, or LIDAR with the vision-based system through filtering techniques like Kalman or particle filters to achieve more robust and reliable localization. Additionally, exploring more neural network architectures may help to improve prediction accuracy. Finally, optimizing the model for deployment on embedded platforms with limited computational resources through techniques such as pruning, quantization, or hardware acceleration could be worked upon.

References

[1] N. Koenig and A. Howard, "Design and use paradigms for Gazebo, an open-source multi-robot simulator," 2004 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS) (IEEE Cat. No.04CH37566), Sendai, Japan, 2004, pp. 2149-2154 vol.3, doi: 10.1109/IROS.2004.1389727.

[2] M. Ismail *et al.*, "Soccer robot localization based on sensor fusion from odometry and omnivision," 2012.

[3] Diederik P. Kingma and Jimmy Ba. *Adam: A method for stochastic optimization.* arXiv preprint arXiv:1412.6980, 2014.

[4] Q. Liu *et al.*, "A self-localization method through pose point matching for autonomous soccer robot based on omni-vision," in *Proceedings of RoboCup Symposium*, 2009.

[5] H. Lu *et al.*, "Robust and real-time self-localization based on omnidirectional vision for soccer robots," 2013.

[6] X. Luo *et al.*, "Robot detection and localization based on deep learning," 2020.

[7] Steve Macenski, Tully Foote, Brian Gerkey, Chris Lalancette, and William Woodall. *Robot Operating System 2: Design, architecture, and uses in the wild.* Science Robotics, vol. 7, no. 66, 2022.

[8] A. Nadiri *et al.*, "IMU-stabilized bird's-eye view localization for humanoid soccer robots," 2025.

[9] M. Ribeiro *et al.*, "Fast computational processing for mobile robots self-localization," 2016.

[10] Y. Tian *et al.*, "Self-localization of humanoid robots with fish-eye lens in a soccer field," 2010.

[11] Y. Watanabe *et al.*, "Model-based self-localization with genetic algorithm for omnidirectional vision," 2020.

[12] M. Sundararajan, A. Taly, and Q. Yan, "Axiomatic attribution for deep networks," in *Proceedings of the 34th International Conference on Machine Learning (ICML)*, vol. 70, pp. 3319–3328, 2017.

[13] D. Tola and P. Corke, "Understanding URDF: A dataset and analysis," *IEEE Robotics and Automation Letters*, vol. 9, no. 5, pp. 4479–4486, 2024.

[14] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788.

[15] R. Ranftl, A. Boedt, and V. Koltun, "Towards robust monocular depth estimation: Mixing datasets for zero-shot cross-dataset transfer," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 3, pp. 1623–1637, 2020.